



Keep Notes

A Google Keep-style notes application on the MERN stack — with JWT authentication, markdown notes, tag-based filtering and per-note version history. Built, tested, and deployed end to end.

MongoDB 7

Express 4

React 18

Node 20

Mongoose 8

Vite 5

Tailwind 3

JWT

Jest + supertest

Docker

Author Muhammad Muttaqi

Context Full-Stack Developer Intern, EyraTech Corporation — NASTP, Pakistan

Repository github.com/muttaqi123/notes-app

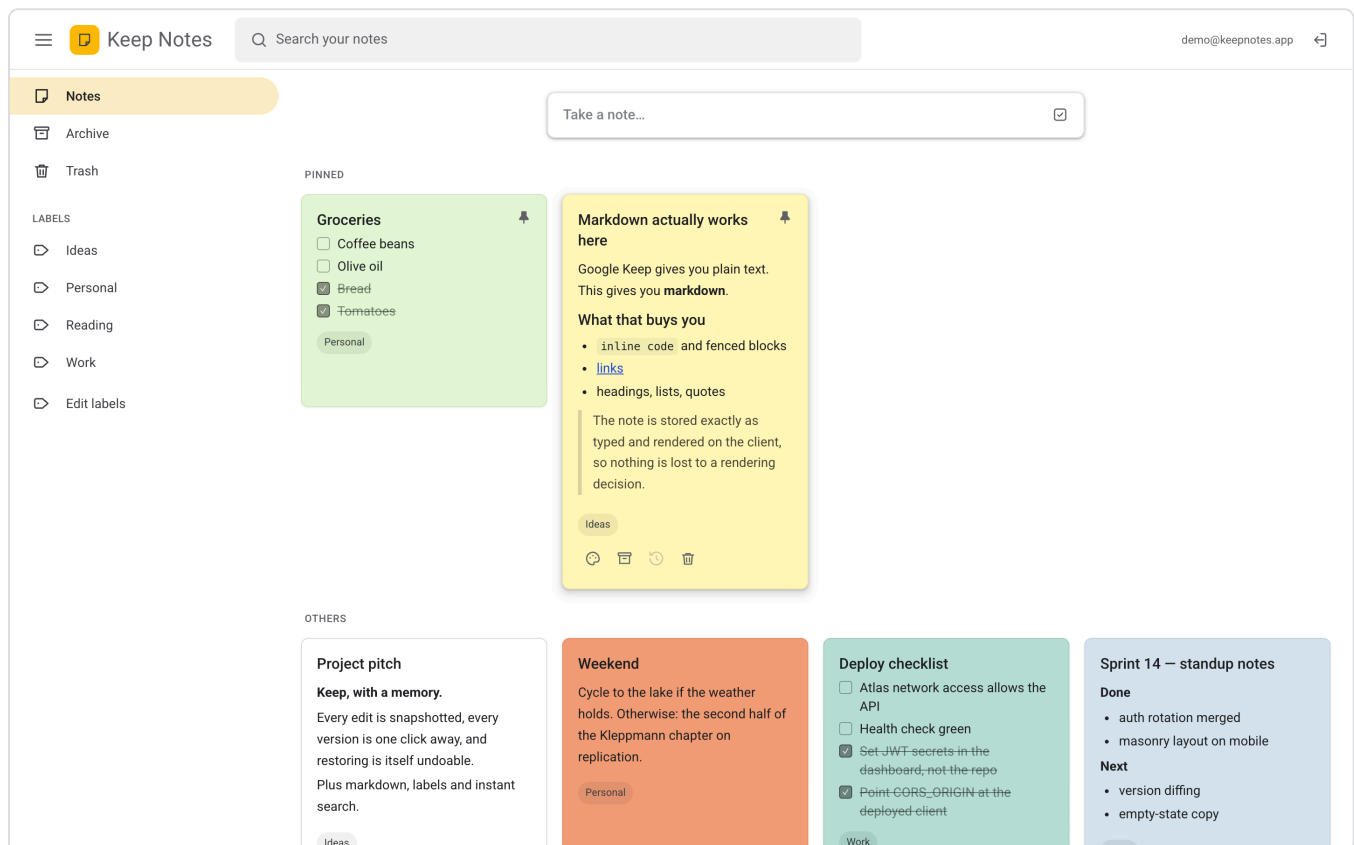
Tests 36 integration tests, all passing

A written walkthrough of what was built, what it was built with, how it was built, and — for every decision an interviewer is likely to ask about — why it went the way it did.

00 What is in this document

Read section 9 before an interview. The rest is the detail behind it.

1. **The brief, and what was delivered** — the feature list, and how each item maps to the app
2. **Tools and software** — every library and program used, and what each one is for
3. **Architecture** — the layering, the folder map, and the life of one request
4. **The data model** — four collections, and why they are shaped that way
5. **Authentication** — access tokens, refresh rotation, and where a token is allowed to live
6. **The three features Keep does not have** — markdown, tags, version history
7. **The front end** — masonry, optimistic updates, instant search
8. **Testing, running and deploying** — what is covered, and how to put it online
9. **Interview questions, with answers** — the twenty most likely, and how to answer them
10. **Limitations and what comes next** — the honest version



The notes board: pinned notes above the rest, masonry columns, colour-coded cards, checklists and markdown side by side, labels in the rail on the left.

01 The brief, and what was delivered

The brief was to extend Google Keep with the things it does not do, while still delivering the Keep-style core end to end.

Extensions beyond Keep

REQUIRED	DELIVERED	WHERE IT LIVES
JWT authentication	Registration and sign-in, 15-minute access tokens, opaque refresh tokens rotated on every use and stored only as hashes, per-device sign-out	<code>services/auth.service.js</code> , <code>utils/tokens.js</code> , <code>middleware/auth.js</code>
Markdown-formatted notes	Notes written in markdown, live preview toggle in the editor, rendered on cards, parsed with <code>marked</code> and sanitised with <code>DOMPurify</code>	<code>lib/markdown.js</code> , <code>NoteEditor.jsx</code>
Tag-based filtering	Labels as first-class documents, attached to notes, listed in the nav rail, each one its own filtered route	<code>models/Label.js</code> , <code>services/labels.service.js</code>
Note version history	Every content edit snapshotted, the full history browsable per note, one-click restore — and the restore is itself undoable	<code>models/NoteVersion.js</code> , <code>VersionHistory.jsx</code>

The Keep-style core

FEATURE	DETAIL
Notes & checklists CRUD	Two note types sharing one document shape; a note converts to a checklist and back without losing the other's content
Colour-coded labels	Eleven note colours from Keep's own palette, plus label chips on every card
Pin	Pinned notes sort into their own section above the rest; pinning an archived note un-archives it, as in Keep
Archive	A separate view; a note can be archived straight from the composer without ever hitting the board
Trash	Soft delete — a trashed note keeps its row and can be restored. Only "empty trash" and "delete forever" actually remove data
Instant client-side search	Filters title, body, checklist items and label names as you type, with no request per keystroke
Responsive React UI	Masonry board from one column on a phone to four on a wide screen; the nav rail slides away under 768px

WORTH SAYING OUT LOUD IN AN INTERVIEW

The starting point was a NestJS scaffold and an untouched Create React App — boilerplate with no logic in it. The brief asked for MERN, and Nest is not the E in MERN, so the honest move was to restructure rather than to bolt Express onto a Nest app. Nothing of value was discarded; the scaffold had never had a line of business logic written into it.

02 Tools and software

Everything used to build, run, test and ship the app.

The stack itself

TOOL	VERSION	WHAT IT DOES HERE, AND WHY THIS ONE
Node.js	20 LTS	The server runtime. ES modules throughout, so <code>import</code> works on both sides of the app and there is no build step on the server at all.
Express	4.21	The HTTP layer. Small and explicit — the routing table <i>is</i> the framework, which keeps the interesting code out of it.
MongoDB	7	The database. A note is naturally one document: a title, a body, an array of checklist items, an array of label references. Storing it as one object is closer to what it is than three joined tables would be.
Mongoose	8.9	The ODM. Adds the schema, validation, indexes and population that the raw MongoDB driver deliberately leaves out.
React	18.3	The UI. Hooks only, no class components; state lives in one custom hook rather than a global store.
Vite	5.4	Dev server and bundler. Instant hot reload, and its dev proxy forwards <code>/api</code> to Express so there is no CORS in development.
React Router	6.28	Client-side routing. Each view — board, archive, trash, one label — is a real URL that survives a refresh.
Tailwind CSS	3.4	Styling. Keep's interface is cards, spacing and colour; utilities express that faster than a stylesheet, and the whole design system fits in one config file.

Libraries doing one specific job

LIBRARY	JOB
<code>jsonwebtoken</code>	Signs and verifies the access token
<code>bcryptjs</code>	Hashes passwords at cost 12 — deliberately slow, so guessing is slow
<code>zod</code>	Validates every request body at the edge; unknown keys are stripped, not passed inward
<code>helmet</code>	Sets the security response headers Express does not set on its own
<code>express-rate-limit</code>	Caps sign-in and registration attempts per IP
<code>cookie-parser</code>	Reads the <code>httpOnly</code> refresh cookie
<code>cors</code>	Allows the deployed client origin, and only that, to send credentialed requests
<code>morgan</code>	Request logging in development
<code>marked</code>	Markdown to HTML
<code>dompurify</code>	Sanitises that HTML before it is ever inserted into the page

Development, testing and operations

TOOL	WHAT IT WAS USED FOR
Jest 29	The test runner, in ESM mode
supertest 7	Drives the real Express app in-process — no port, no listener, no HTTP client to configure
mongodb-memory-server 10	A real MongoDB started in memory for the test run. Real indexes, real validation, real ObjectIds — and nothing to install on a clean checkout. It doubles as the development fallback when no <code>MONGODB_URI</code> is set.
Docker + Docker Compose	Runs <code>mongo:7</code> locally, so development matches production instead of relying on the in-memory stand-in
Git & GitHub CLI	Version control; the repository was created and pushed with <code>gh repo create</code>
Puppeteer + headless Chrome	Captured the screenshots in this document from the running application, and rendered this PDF
concurrently	<code>npm run dev</code> starts the API and the client together with one command

CHOSEN AGAINST, ON PURPOSE

- **Redis** — nothing to cache and no cross-process state. Refresh tokens live on the user document, which is one query away regardless.
- **Redux / Zustand** — one screen owns the note state. A store would add indirection without removing any.
- **TypeScript** — a real trade-off. It would catch shape errors the tests catch later. The integration suite covers the API boundary, which is where those errors actually bite.
- **An ODM-free driver** — the schema, the indexes and `populate` would all have had to be hand-rolled.

03 Architecture

THE ONE RULE EVERYTHING ELSE FOLLOWS

Business logic lives in `server/src/services/`, and nothing in that folder imports `Express`. A service takes plain values and returns plain values. It never sees a `Request`, never sets a status code, never builds a response.

Two things follow from that, and both are worth saying in an interview. First, every rule about a note is testable directly, without an HTTP client. Second, a second client — a mobile app, a CLI, a chatbot — would reuse the logic unchanged, because none of it is entangled with HTTP.

The controllers are deliberately three lines each: read the request, call a service, send the result. If one ever grows an `if`, that `if` belongs in a service.

The life of one request

```
PATCH /api/notes/:id          "add a line to this note"
├─ helmet, cors, express.json, cookieParser    cross-cutting middleware
├─ requireAuth                                verifies the JWT → req.userId
├─ validate(updateNoteSchema)                  zod parses the body; unknown keys stripped
├─ notesController.update                      3 lines: read, call, respond
│   └─ notesService.updateNote(userId, noteId, patch)
│       └─ load the note WHERE _id AND owner    ← ownership in the query
│       └─ did this patch touch content?
│       └─ if yes: write a NoteVersion snapshot of the OLD state
│       └─ if yes: version += 1
│       └─ save
└─ errorHandler                              the only place an error becomes a status code
```

Folder map

```
notes-app/
├─ server/
│   └─ src/
│       ├── app.js                builds the Express app – no listener, no DB
│       ├── index.js              connects the DB and listens
│       ├── config/               env parsing, DB connection
│       ├── models/               User, Note, NoteVersion, Label
│       ├── services/             ALL BUSINESS LOGIC – no Express in here
│       ├── controllers/          read the request, call a service, respond
│       ├── routes/               paths, auth, zod schemas
│       ├── middleware/           auth, validation, the one error handler
│       └─ scripts/seed.js        a demo account worth screenshotting
├─ tests/                         36 integration tests
└─ client/
    └─ src/
        ├── api/client.js         fetch wrapper, token refresh, one retry
        ├── context/              auth state
        ├── hooks/useNotes.js     data, optimistic updates, instant search
        ├── components/           Shell, NoteCard, Composer, NoteEditor, ...
        └─ pages/                 AuthPage, NotesPage
```

WHY APP.JS AND INDEX.JS ARE SEPARATE FILES

`app.js` builds the Express app and does nothing else — it opens no port and touches no database.

`index.js` connects the database and starts listening. That split is what lets the test suite mount the entire API against an in-memory MongoDB and drive it in-process with supertest: no port to pick, no race between "the server is starting" and "the test is running", and no way for two test files to collide on a port.

04 The data model

Four collections. Every one of them carries an owner.

```
users      { name, email(unique), passwordHash,
            refreshToken: [{ tokenHash, expiresAt, createdAt }] }

labels     { owner, name, color }
            unique index on (owner, name)

notes      { owner, title, body, type: 'note'|'checklist',
            items: [{ text, checked }],
            color, pinned, archived, trashedAt,
            labels: [ObjectId → labels], version }
            index on (owner, pinned desc, updatedAt desc)

noteversions { note, owner, version, title, body, type, items,
              color, labels, reason: 'edit'|'restore', createdAt }
              index on (note, version desc)
```

Five decisions in that schema

One document per note, not a notes table joined to an items table

A checklist item has no meaning apart from its note, is never queried on its own, and is always read and written with its parent. That is the exact case where embedding beats referencing — one read gets the whole note.

Labels are referenced, not embedded

The opposite call, for the opposite reason. A label is shared across many notes and has a life of its own: renaming it must change it everywhere at once. Embedded copies would need a fan-out write to every note that used it.

Versions are their own collection

Versions are written on every edit and read almost never. As an array on the note they would make every note document grow without limit — and that document is on the hot path, since the board reads all of them at once. In their own collection they cost nothing until someone opens the history.

trashedAt is a date, not a boolean

A boolean answers "is it in the trash". A timestamp also answers "since when", which is what an auto-purge after thirty days would need. It costs the same to store and it answers a question a boolean cannot.

version is stored, not derived from counting snapshots

The number on a note is not derivable from counting its snapshots, because a restore adds a snapshot without being an ordinary edit. Storing the counter keeps the numbering monotonic and stable, which is what the history list displays.

THE INDEX THAT MATTERS

{ owner: 1, pinned: -1, updatedAt: -1 } is exactly the board query: this user's notes, pinned first, newest first. A compound index in the same order as the sort means MongoDB walks it and returns rows already ordered, instead of loading the set and sorting it in memory.

05 Authentication

Two tokens with two different jobs. This is the section interviewers push hardest on, so it is the one worth knowing cold.

	ACCESS TOKEN	REFRESH TOKEN
What it is	A signed JWT	48 random bytes — not a JWT
Lifetime	15 minutes	30 days
Lives in	A JavaScript variable, in memory	An <code>httpOnly</code> cookie
Sent as	<code>Authorization: Bearer ...</code>	Automatically, by the browser
Stored server-side	Not at all — it is verified by signature	As a SHA-256 hash on the user document
Revocable	No, but it expires in 15 minutes	Yes, immediately

Why the access token is not in localStorage

`localStorage` is readable by any script running on the page. One XSS bug — a bad dependency, an unsanitised note — and the token is gone. Held in a module variable it dies with the tab and is never in a place a script can enumerate. The cost is that a page refresh loses it, which is precisely the job the refresh cookie does.

Why the refresh token is not a JWT

Nothing about it needs reading without a database round trip, and a JWT can be forged by anyone who learns the signing secret. A random opaque string cannot be forged at all — it either matches a stored hash or it does not. Only the hash is stored, so a leaked database dump contains no usable token.

Rotation, and why it is the interesting part

```
POST /api/auth/refresh
├ hash the presented token, look up the user by that hash
├ expired or unknown? → 401, and delete the entry
├ DELETE the presented token from the user ← the rotation
├ issue a NEW refresh token and store its hash
└ return a new access token, set the new cookie
```

A refresh token is good for exactly one use. If one is stolen and used, the real user's next refresh fails and they are signed out — the theft becomes visible instead of silent. Tokens are stored per device, so signing out on one machine does not sign out the others, and expired entries are pruned on every issue so the array cannot grow without bound.

One subtlety in the client

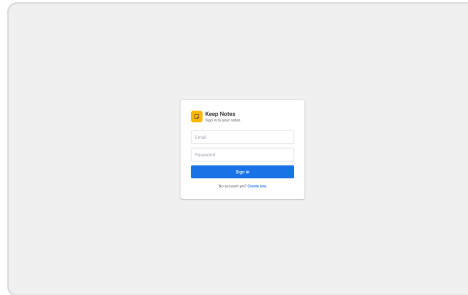
Because rotation invalidates the old token, five requests firing on page load must not trigger five refreshes — four would be rejected and sign the user out. The API client keeps one in-flight refresh promise and lets every caller await it.

The rest of the surface

- **Identical failures.** A wrong password and an unknown email return the same message, and the code does the same work either way — returning early on an unknown email leaks it through timing. There is a test

asserting the two messages match.

- **Ownership inside the query.** Every note lookup filters on `owner` in the query itself, so there is no window where a note is loaded and only then found to belong to someone else. Cross-account access returns **404**, not **403** — a 403 confirms the id exists.
- **Secrets fail closed.** `config/env.js` refuses to start in production without real secrets, rather than falling back to a development default. A default shipped to production is a published private key.
- **Rate limiting** on sign-in and registration, and a 1 MB body cap.



Sign-in. Registration is the same form with one more field — two nearly identical files would drift.

06 The three features Keep does not have

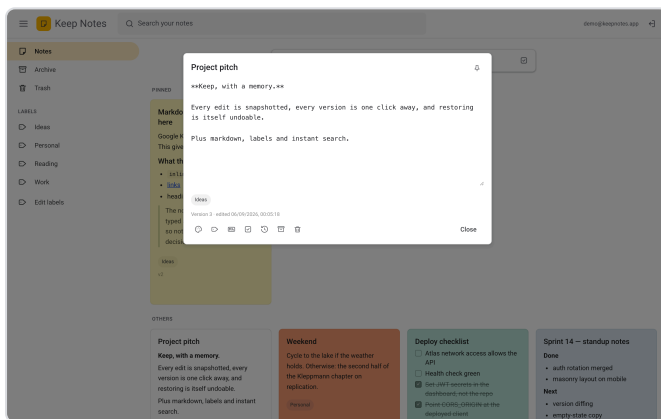
Markdown notes

The body is stored exactly as typed. Rendering happens on the client, in two steps that must never be separated:

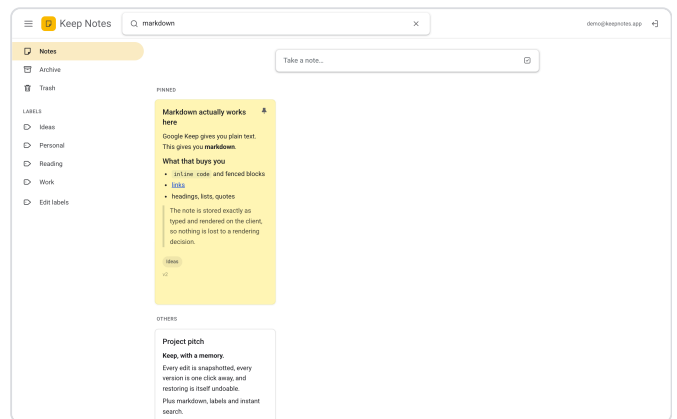
```
const html = marked.parse(source);           // markdown → HTML
return DOMPurify.sanitize(html, { ... });    // HTML → safe HTML
```

The sanitiser is not optional. A note is user input rendered back as HTML, so without it a note containing `<img onerror=...>` is stored XSS — against the person who wrote it, and against anyone the note is ever shared with. The allow-list also blocks `javascript:` URLs: a link that runs code is not a link.

Storing the source rather than the rendered HTML means the user's text is never lost to a rendering decision, and the renderer can be swapped later without a migration.



The editor, showing raw markdown in a monospace field.



Instant search, filtering the notes already in memory.

Tag-based filtering

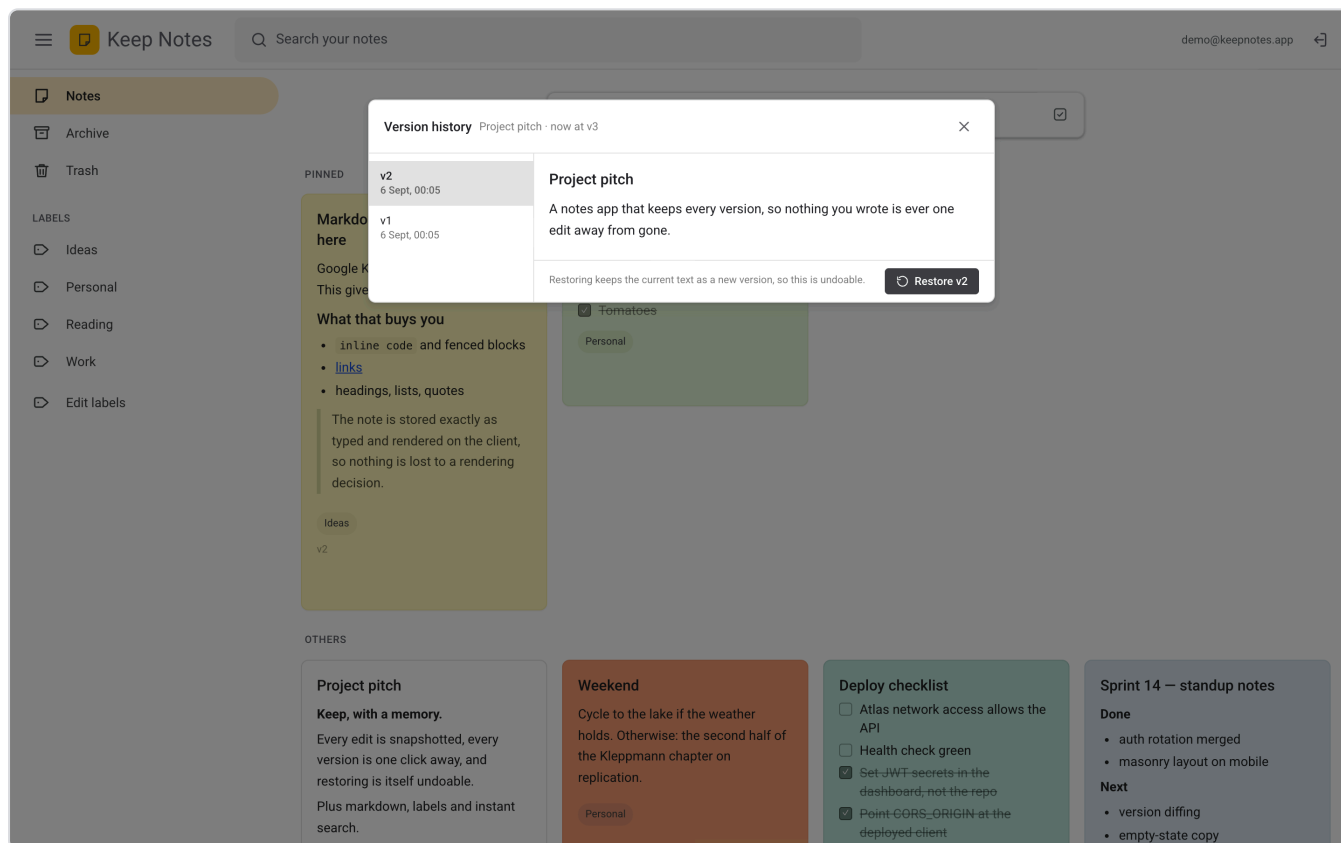
Labels are documents, not strings on a note. That buys three things a string array cannot: a rename that propagates everywhere at once, a uniqueness constraint per user (`unique (owner, name)`) — two users may both have "Work"), and a colour of their own.

Two behaviours worth pointing at. Label ids sent by a client are filtered against that user's own labels before being attached, so nobody can attach a stranger's label by guessing an id. And deleting a label **detaches** it from its notes rather than deleting them — losing a tag must never be a way to lose the writing under it.

Version history

Every note carries a `version` number. On an edit that changes **content**, the server writes a snapshot of the state *before* the edit and increments the number.

DECISION	REASONING
Snapshot the old state, before writing the new one	If the write of the new state fails, the history still describes reality.
Content changes only	Pinning, archiving or recolouring is not an edit to the note's text. A history full of "you pinned this" is a history nobody reads.
Compare before writing	A PATCH that sets the body to what it already was creates no version. Opening a note to read it must not add to its history.
Save on close, not on keystroke	One editing session becomes one version — a history a person can read. Per-keystroke versioning produces a history nobody can.
Restoring is itself versioned	Restoring v1 snapshots the current text first, so going back is never how you lose the newer draft.
Deleting forever deletes the history too	No orphaned snapshots of a note that no longer exists.



Version history: every past state on the left, the selected one rendered on the right, and a restore that says in the interface itself that it is undoable.

07 The front end

The masonry board is CSS, not JavaScript

Notes have wildly different heights. CSS multi-column with `break-inside: avoid` handles that natively — no resize observer, no measuring pass, no layout recalculation on every keystroke. The column count is a responsive utility: one on a phone, four on a wide screen.

Search is client-side, deliberately

The API supports `?q=` and it is tested. The UI does not use it. Filtering the notes already in memory is instant, costs no request per keystroke, and works with a flaky connection. The server-side search stays for the day a user has more notes than one page can hold — at which point the UI switches to it and nothing else changes.

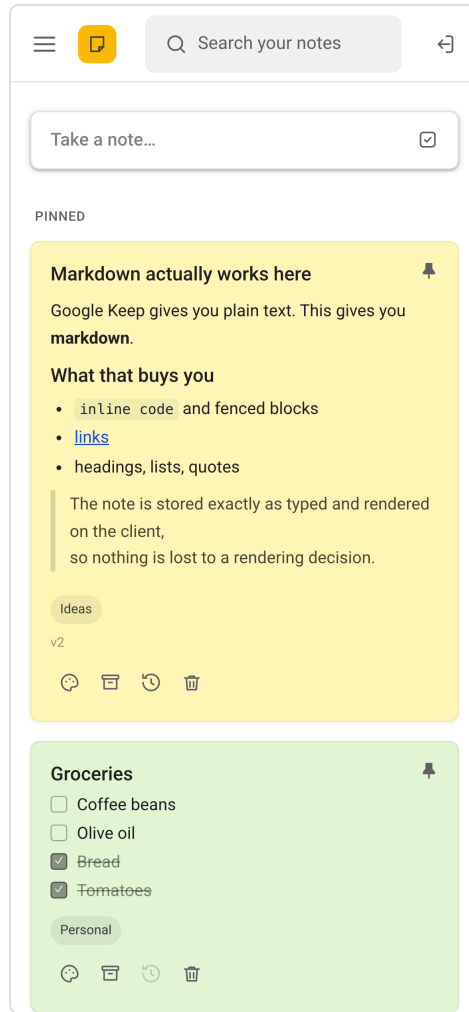
The server-side version escapes regex metacharacters before building the query, so searching for `$40.00` searches for that text rather than executing it as a pattern. There is a test for exactly that.

Writes are optimistic

Clicking pin moves the card immediately; the server response replaces the local copy when it lands. On failure the previous state is put back and the error is shown — a pin that silently un-pins itself is worse than one that says why.

Small things that took thought

- **A stale-response guard.** Switching views fires a request; a slow answer for a view you already left must not overwrite the one you are looking at. A request counter drops anything but the newest.
- **An empty note is discarded,** on the server as well as in the UI, so a client that always POSTs on close cannot litter the database with blank cards.
- **Checklist keyboard behaviour.** Enter adds the next item and focuses it; Backspace on an empty item removes it and focuses the one above. Those two keystrokes are what make a list feel like a list rather than a form.
- **The page is locked behind an open dialog,** restoring the previous overflow value rather than a hard-coded default, so two stacked dialogs cannot unlock the page between them.
- **Menus close on Escape as well as on an outside click** — a menu you can only close with a mouse is a trap for anyone not using one.



390 px: one column, the nav rail slid away, action buttons always visible because there is no hover on a touch screen.

08 Testing, running and deploying

36 integration tests, and why they are integration tests

Every test drives the real Express app through supertest against a real MongoDB started in memory. Not unit tests with mocked models: the bugs worth catching here live in the seams — does the ownership filter actually filter, does the unique index actually reject, does a PATCH actually create a version. A mocked model cannot answer any of those.

FILE	TESTS	COVERS
<code>auth.test.js</code>	8	Registration, duplicate email, weak passwords, identical failure messages, the httpOnly cookie, refresh rotation, protected routes
<code>notes.test.js</code>	13	CRUD, markdown stored verbatim, checklists, pin/archive independence, sort order, trash and restore, search including a regex metacharacter, and cross-account isolation
<code>versions.test.js</code>	9	Snapshot on content change, no snapshot on pin/archive, no snapshot on a no-op PATCH, restore, restore-is-undoable, privacy, cascade on delete
<code>labels.test.js</code>	6	Uniqueness per user, filtering, a stranger's label being dropped, delete detaches without deleting notes, rename propagates

ONE BUG THE TESTS CAUGHT IMMEDIATELY

The first run failed 35 of 36 with `429 Too Many Requests`. The rate limiter was written as `limit: isTest ? 0 : 30` — but in express-rate-limit v7, a limit of zero means *allow zero requests*, not *disabled*. The fix is `skip: () => isTest`. It is a good example to give when asked about debugging: the symptom named the cause, and the mistake was assuming an API's semantics instead of checking.

Running it

```
git clone https://github.com/muttaqi123/notes-app.git
cd notes-app
npm run install:all
npm run dev          # API on :4000, client on :5173
```

With `MONGODB_URI` unset the server starts an in-memory MongoDB on boot, so the app runs on a clean machine with no database installed — which is exactly what you want when someone else is trying your project. For data that persists: `docker compose up -d`, then `npm run seed` for a demo account.

Deploying it

PIECE	WHERE	CONFIGURATION
Database	MongoDB Atlas, free tier	Region near the users; the API's egress added to Network Access
API	Render (or any Node host)	Root <code>server</code> , build <code>npm ci</code> , start <code>npm start</code> , health check <code>/api/health</code> . <code>MONGODB_URI</code> , both JWT secrets and <code>CORS_ORIGIN</code> set in the dashboard.
Client	Vercel or Netlify	Root <code>client</code> , build <code>npm run build</code> , output <code>dist</code> , <code>VITE_API_URL</code> pointing at the API

TWO DEPLOYMENT DETAILS WORTH MENTIONING

- **The SPA rewrite.** `vercel.json` rewrites every path to `index.html`. Without it, refreshing on `/archive` is a 404 from the CDN — the CDN looks for a file, and the router never gets a chance to run.
- **A config file is not a guarantee.** `render.yaml` only applies to a service created *from* a Blueprint; a service created by hand in the dashboard ignores it entirely. That is why the secret check lives in `config/env.js` — a security default has to be safe in code, so that forgetting the deployment file is harmless.

09 Interview questions, with answers

The twenty most likely, shortest defensible answer first.

On the stack

Why MongoDB and not Postgres?

A note is naturally one document — title, body, an array of items, an array of label references — and it is always read whole. Embedding checklist items removes a join that would exist purely as a storage artefact. If the app grew reporting across users, or transactions spanning collections, that calculus would change.

Why Mongoose over the raw driver?

Schema, validation, index declaration and `populate`. The driver leaves all four to you, and hand-rolled versions of them are exactly where inconsistent data creeps in.

Why Express and not NestJS?

The brief said MERN, and Express is the E. Nest's structure is valuable on a large team; here the same separation is achieved by the services rule, without the framework overhead.

Why no TypeScript?

A real trade-off, and I would add it on a team. The integration suite covers the API boundary, which is where shape errors actually bite. On a codebase this size the tests were the higher-value spend.

On authentication

Walk me through your JWT flow.

Sign-in returns a 15-minute access JWT in the response body and sets an `httpOnly` refresh cookie. The client keeps the access token in memory and sends it as a Bearer header. On a 401 it refreshes once and retries transparently. Refresh rotates: the presented token is deleted as it is used and a new one issued.

Where do you store the token, and why not localStorage?

In memory. `localStorage` is readable by any script on the page, so a single XSS bug is a stolen session. In memory the token dies with the tab, and the `httpOnly` cookie — which JavaScript cannot read at all — is what survives a reload.

A JWT can't be revoked. How do you sign someone out?

You revoke the refresh token, which is server-side state, and the access token expires within fifteen minutes. That is the trade: a short window of un-revocable access in exchange for stateless request verification.

Why rotate refresh tokens?

So a stolen one is good for a single use, and so the theft becomes visible — the real user's next refresh fails and signs them out, instead of an attacker holding a valid 30-day session silently.

How do you stop one user reading another's note?

The owner is part of the query, not a check after loading: `findOne({ _id, owner })`. There is no window in which a note is loaded and only then found to belong to someone else. It returns 404 rather than 403, because a 403 confirms the id exists. Two tests assert it.

On the features

How does version history work?

Each note has a version counter. On an edit that changes content, the server writes a snapshot of the previous state into a separate collection and increments the counter. Non-content changes — pin, archive, colour — do not version. Restoring snapshots the current state first, so a restore is undoable.

Why a separate collection and not an array on the note?

Versions are written often and read almost never. An array would make every note document grow without bound, and that document is on the hot path — the board loads all of a user's notes at once. In its own collection the history costs nothing until someone opens it.

Isn't storing full snapshots wasteful? Why not diffs?

Yes, and deliberately. Notes are small, restoring a snapshot is a copy while restoring a diff chain is a replay, and a corrupt snapshot loses one version where a corrupt diff loses everything after it. Diffs would be right for large documents; these are not.

You render user content as HTML. What about XSS?

Markdown is parsed with `marked`, then the resulting HTML goes through DOMPurify with an explicit tag and attribute allow-list, and a URL pattern that blocks `javascript:`. Without the sanitiser a note containing `<img onerror>` would be stored XSS.

Why is search client-side?

It is instant, it fires no request per keystroke, and it works offline. The server-side search exists and is tested, for when a user has more notes than one page holds — switching to it is a change in one hook.

On the engineering

How is the code organised?

Business logic in `services/`, which imports nothing from Express. Controllers are three lines: read, call, respond. That is what makes the logic directly testable and what would let a second client reuse it unchanged.

What does an optimistic update roll back?

The whole previous notes array, captured before the local change. On failure it is restored and the error surfaced, so the UI never diverges silently from the server.

How do you test without a database?

`mongodb-memory-server` starts a real MongoDB in memory for the run — real indexes, real validation, real ObjectId's. Collections are wiped between tests, and `app.js` is separate from `index.js` so supertest mounts the app with no port and no listener.

Tell me about a bug you hit.

Thirty-five of thirty-six tests failed with 429. The rate limiter had `limit: isTest ? 0 : 30`, and in express-rate-limit v7 a limit of zero means allow nothing, not disabled. The fix was `skip`. The lesson was that I had assumed an API's semantics rather than checking them.

What would you do about a race — two tabs editing one note?

Today the last write wins, and the history means nothing is lost. The fix is a version check on the PATCH: reject if the client's version is not current, and let the UI offer the newer text. The counter for it already exists.

How would this scale?

The board query is a single indexed read per user and MongoDB shards naturally on `owner`. The first real limit is loading all notes at once — that needs pagination, and search moves to the server index at the same time. The API is stateless apart from refresh tokens in the database, so it scales horizontally as it stands.

What is the weakest part?

No client-side tests. The server is covered thoroughly; the React layer is verified by hand. I would add React Testing Library for the editor's save-on-close logic and the optimistic-update rollback, since those are the two places where a regression would be silent.

10 Limitations, and what comes next

Knowing what is missing reads better than claiming nothing is.

Known limitations

- **No client-side tests.** The API has 36; the React layer has none.
- **The board loads every note at once.** Fine at hundreds, not at ten thousand. Pagination is the fix, and search would move server-side with it.
- **Last write wins.** Two tabs editing one note will have one overwrite the other. The history means nothing is truly lost, but the conflict is not surfaced.
- **The trash never empties itself.** `trashedAt` exists precisely so a scheduled purge can be added; it has not been.
- **No images or drawings.** Keep has both. That needs object storage and is a project of its own.
- **No collaborators.** Every model has an `owner`, which is a deliberately narrow choice; sharing would need a permission model rather than a single field.
- **No password reset.** There is no mail provider wired up, and a reset flow without one is theatre.

What I would build next, in order

1. **Optimistic-concurrency on PATCH** — send the version, reject a stale write, offer the newer text. Small, and it closes a real correctness gap.
2. **Client tests** for the two places a regression would be silent: save-on-close and optimistic rollback.
3. **Pagination plus server-side search**, which are one change, not two.
4. **A visual diff between versions** — the data is already there; only the rendering is missing.
5. **A scheduled trash purge**, using the timestamp already stored.

IF YOU REMEMBER ONE THING FROM THIS DOCUMENT

The services rule is the spine of the project: every rule about a note lives in one framework-free folder, the HTTP layer is three lines per route, and the tests drive the real app against a real database. Every other decision here — embedding items but referencing labels, snapshots over diffs, a token in memory rather than in `localStorage`, search on the client — is a trade-off with a reason attached, and the reason is the part worth saying out loud.